WTQ Build Track Participant Guide

Utility Bill Decoder

1. The challenge

Most people find their electricity bill hard to follow. It mixes tariff slabs, taxes, fuel adjustments and many line items, and each provider lays it out differently. Your challenge is to build an **AI-powered assistant that reads a bill, pulls out its details, and explains them in plain language**.

The input is an **electricity bill** from K-Electric, LESCO or IESCO, as a scanned image, screenshot or photo. Layouts vary: some bills are tilted or partly in Urdu, and names, addresses and ID numbers are covered with grey boxes.

This guide explains the data, output formats, submission requirements and scoring.

1.1 The two levels

The challenge has two levels. **Both levels are required.**

Level	Task	Required output
Level 1: Extraction	Extract non-identifying billing details, readings, charges, taxes, and payable amounts.	One JSON object per bill
Level 2: Explanation and questions	Answer customer questions using the bill. Some answers require calculations or estimates.	One text answer per bill and question

2. Schedule and links

Key times for the day, and the links you need to download the data and submit your work.

Time	What to do
10:30 AM	Download the five training bills, set up an API key for one of the allowed models (section 9), and start building your solution.
12:30 PM	Download the 10 test bills and two CSV templates. Run your solution to generate the completed CSV files.
By 1:30 PM	Upload one ZIP containing your results and source code. A demo video is optional.

Resource	Link
Data folder	https://tinyurl.com/wtq26 (Google Drive)
Submission portal	https://build.womentechquest.com/submit

You can submit only once. Re-submission is not allowed. Check the ZIP before uploading it.

Find a section: LLMs and tools | Data | Level 1 | Level 2 | What to submit | Scoring | API keys

3. Allowed LLMs and tools

Your solution may call only the LLMs listed below. **Using any other LLM leads to disqualification.** Section 9 lists each model's ID, rate limits and price, and explains how to get an API key: AI models and API keys.

- Claude Haiku 4.5
- Gemini 3.5 Flash-Lite
- Gemini 3.1 Flash-Lite
- GPT-4.1 mini
- GPT-5 mini

Other rules:

- **AI assistance is allowed.** You may use AI coding assistants and chatbots to help write your code. List every AI tool you used in the README's AI usage section.
- You may use any other tools and libraries that comply with these rules.
- **Local OCR libraries are allowed** to help read bills, such as Tesseract or PaddleOCR.
- **Invoice or receipt extraction agents and Azure services are prohibited**, including Azure prebuilt-invoice, Textract AnalyzeExpense, and Mindee.
- List every model and service you use in `README.md`. The list must match your code.

4. Data

You receive training bills to build and test your solution with, and test bills that your submission is scored on. Both datasets will be available in the same Google Drive folder: **https://tinyurl.com/wtq26**. Bill images are in `.png` or `.jpg` format.

4.1 Training data (10:30 AM)

The training data has five sample bills for you to explore and to build your solution with. They show the different bill layouts you will meet in the test.

```text
train/
└── bills/
    ├── IESCO_0004.jpg
    ├── KESC_0004.png
    ├── KESC_0008.png
    ├── LESCO_0005.png
    └── LESCO_0006.png
```

The training data has **bills only, no answers**. To see what correct output looks like, use this guide:

- **Level 1:** section 5.5 walks through one sample bill (`KESC_0008`) and its correct JSON.
- **Level 2:** section 6.2 shows sample questions and good answers for sample bill `KESC_0008`.

4.2 Test data (12:30 PM)

The test data has 10 new bills and two CSV templates. Your solution reads the bills and generates the completed CSV files. These results are what your submission is scored on.

```text
test/
├── bills/              10 bill images
└── csv/
    ├── level1.csv      columns: bill_id, json  (json is empty)
    └── level2.csv      columns: bill_id, question_id, question, answer  (answer is empty)
```

- `bill_id` is the bill image's file name without its extension: `KESC_0002` means `bills/KESC_0002.png`. K-Electric file names use `KESC`, but the JSON `provider` value is `KE`.
- `level1.csv` has one row per test bill (10 rows). Your solution writes the Level 1 JSON for that bill into the `json` column (section 5).
- `level2.csv` has one row per bill and question. Every bill gets the same 12 questions (`Q1` to `Q12`), so there are 120 rows. Your solution writes its answer into the `answer` column (section 6).

This is how the two CSV files look when you receive them (the real `level2.csv` has the full question text):

```level1.csv
bill_id,json
IESCO_0002,
KESC_0002,
...
LESCO_0008,
```

```level2.csv
bill_id,question_id,question,answer
IESCO_0002,Q1,<question 1>,
IESCO_0002,Q2,<question 2>,
...
LESCO_0008,Q12,<question 12>,
```

5. Level 1: Extract bill details

For each bill, your solution returns **one JSON object** with the bill's details. Include every field shown below, even when its value is `null`.

5.1 JSON output format

Every Level 1 output follows this structure. It is a **type template**, not a completed answer: your solution replaces the placeholders with values read from the bill. Use numbers for numeric fields, not quoted strings. Section 5.4 explains each charge and tax type.

```json
{
  "provider": "KE | LESCO | IESCO",
  "tariff": "string",
  "sanctioned_load_kw": "number",

  "bill_month": "YYYY-MM",
  "reading_date": "YYYY-MM-DD",
  "issue_date": "YYYY-MM-DD",
  "due_date": "YYYY-MM-DD",

  "previous_reading": "number",
  "current_reading": "number",
  "units_consumed": "number",

  "charges": [
    {
      "type": "energy | fixed | fpa | quarterly_adjustment | surcharge | meter_rent |
               subsidy | other",
      "amount": "number"
    }
  ],
  "total_charges": "number",

  "taxes": [
    {
      "type": "gst | electricity_duty | income_tax | municipal_tax | other_tax",
      "amount": "number"
    }
  ],
  "total_taxes": "number",

  "current_bill": "number",
  "arrears": "number",
  "payable_within_due_date": "number",
  "payable_after_due_date": "number"
}
```

5.2 General extraction rules

These rules apply to every field in the JSON.

- **Extract values as printed.** Do not recalculate, round, or correct bill amounts.
- **Amounts are numbers in PKR.** Remove currency labels and commas: `Rs. 19,964` becomes `19964`.
- **Credits and subsidies are negative.** For example, `38209CR` becomes `-38209`. Record a subsidy as a deduction.

- **Use the required date formats.** `22nd Sep 2026` becomes `2026-09-22`; `MAR 2026` becomes `2026-03`.
- **Use `null` for unavailable values:** not printed, blank, covered by a grey box, or unreadable. A correct `null` is scored as correct. Do not guess. Use `[]` for an empty list.
- **Exclude personal identifiers.** Do not return names, addresses, CNIC, account numbers, reference numbers, consumer numbers, or meter numbers.

5.3 What to return for each field

The table below says what to return for each field. The same schema applies to every layout: K-Electric, LESCO and IESCO bills. Labels and positions may differ.

Field	What to return
`provider`	One of `KE`, `LESCO`, or `IESCO`.
`tariff`	The tariff code exactly as printed, such as `A1-R` or `A-1a(01)`.
`sanctioned_load_kw`	Sanctioned load in kW, labelled Sanc Load, S.Load, or San Load.
`bill_month`	The billing month in `YYYY-MM` format.
`reading_date`, `issue_date`, `due_date`	The printed dates in `YYYY-MM-DD` format. The due date is the last date to pay without a late payment surcharge.
`previous_reading`, `current_reading`	The printed meter readings.
`units_consumed`	The units billed for the current month, as printed.
`charges`	One entry per line in the charges section. Skip blank lines and lines printed as `0`. Keep separate entries when the same type appears more than once.
`total_charges`	The printed charges subtotal, or `null` if no subtotal is printed.
`taxes`	One entry per line in the tax or government section. Classify a line by its section, not its name. If only a combined tax amount is printed, return `[]` and put that amount in `total_taxes`.
`total_taxes`	The printed tax subtotal, or `null` if no subtotal is printed.
`current_bill`	The printed amount for this month's bill.
`arrears`	Previous dues carried forward. Keep the printed sign; return `null` if blank.
`payable_within_due_date`	The amount payable by the due date.
`payable_after_due_date`	The amount payable after the due date. If several late-payment amounts are shown, use the highest.

5.4 Charge and tax types

Choose a `type` from the tables below for each entry. **The printed section determines whether a line belongs in `charges` or `taxes`.**

5.4.1 Charge types

Use one of these types for each entry in `charges`.

Type	Use for
`energy`	Cost of units used, such as Variable Upto ... Units, Cost of Electricity, or Energy Charges.
`fixed`	Fixed charges.
`fpa`	Fuel price adjustment. Use `fpa` for both FPA and FCA.
`quarterly_adjustment`	Quarterly tariff adjustment, such as QTA or Uniform Quarterly Adjustment.
`surcharge`	Surcharges, such as Additional Surcharge (PHL) or F.C Surcharge.
`meter_rent`	Meter rent or service rent.
`subsidy`	Government subsidy or relief lines. Record the amount as negative.
`other`	Any other charge line.

5.4.2 Tax types

Use one of these types for each entry in `taxes`.

Type	Use for
`gst`	Sales tax or GST, including GST on FPA.
`electricity_duty`	Electricity duty, often labelled ED.
`income_tax`	Income tax.
`municipal_tax`	Municipal or local charges printed under taxes, such as KE MUCT (KMC).
`other_tax`	Any other tax line, such as extra tax, further tax, or TV fee.

5.5 Worked example: KESC_0008

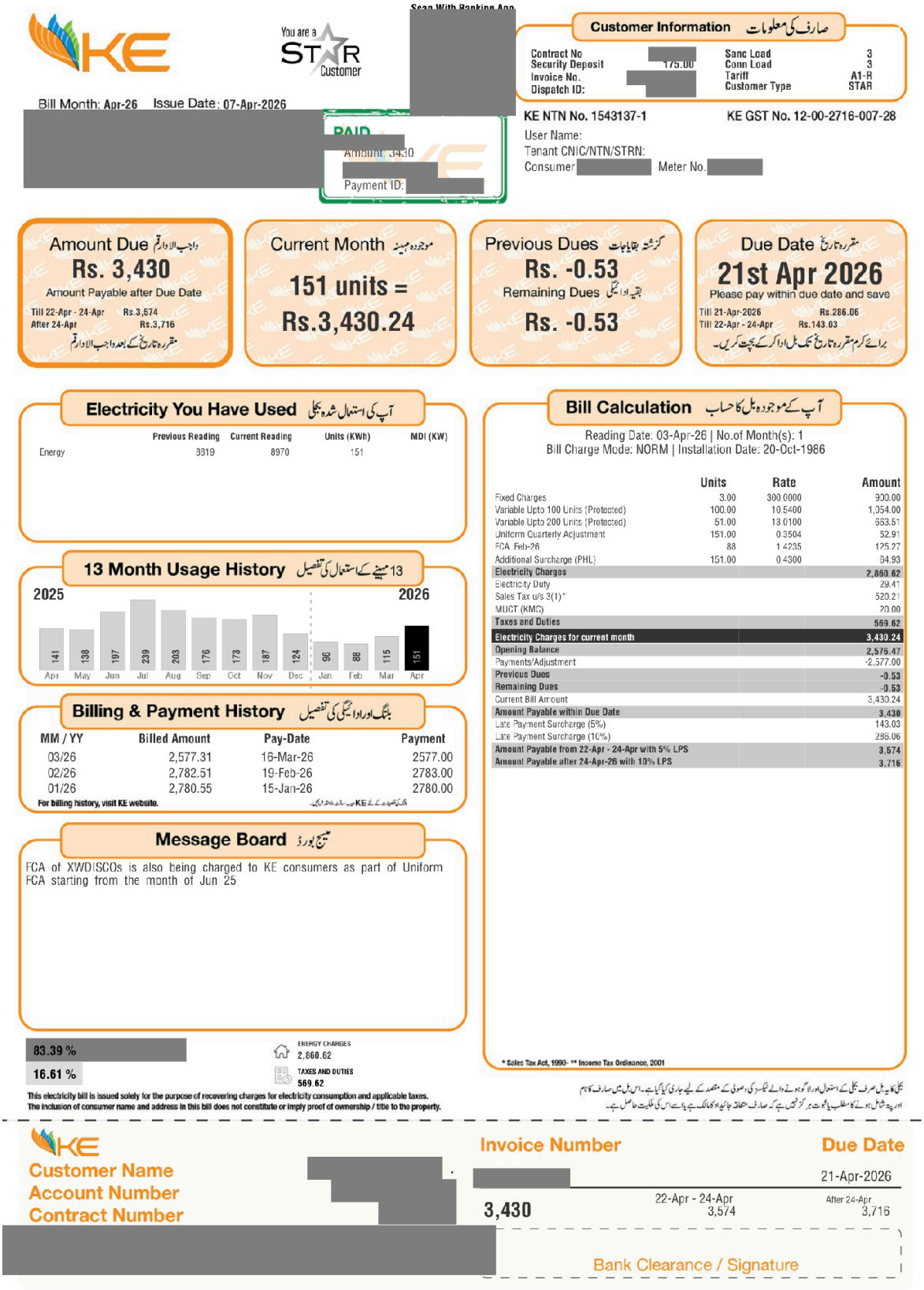
Section 5.6 shows training bill `KESC_0008` and section 5.7 gives its complete Level 1 JSON. Use them together to see how printed values map to fields.

Printed on the bill	JSON field	Value
Current month units	`units_consumed`	151
Electricity Charges subtotal	`total_charges`	2860.62
Taxes and Duties subtotal	`total_taxes`	569.62
Previous Dues	`arrears`	-0.53
Amount payable after 24 Apr	`payable_after_due_date`	3716

5.6 The sample bill

This is the training bill used in the worked example. Personal identifiers are covered with grey boxes.

Training image `KESC_0008.png`



KE

You are a STAR Customer

Scan With Banking App

Bill Month: Apr-26 Issue Date: 07-Apr-2026

[redacted]

PAID

Amount: 3430

Payment ID: [redacted]

Customer Information صارف کی معلومات

Contract No	[redacted]	Sanc Load	3
Security Deposit	175.00	Conn Load	3
Invoice No.	[redacted]	Tariff	A1-R
Dispatch ID:	[redacted]	Customer Type	STAR

KE NTN No. 1543137-1 KE GST No. 12-00-2716-007-28

User Name:

Tenant CNIC/NTN/STRN:

Consumer [redacted] Meter No. [redacted]

Amount Due واجب الادا رقم
Rs. 3,430
Amount Payable after Due Date
Till 22-Apr - 24-Apr Rs.3,574
After 24-Apr Rs.3,716
مقررہ تاریخ کے بعد واجب الادا رقم

Current Month موجودہ مہینہ
151 units =
Rs.3,430.24

Previous Dues گزشتہ بقایاجات
Rs. -0.53
Remaining Dues بقیہ ادائیگی
Rs. -0.53

Due Date مقررہ تاریخ
21st Apr 2026
Please pay within due date and save
Till 21-Apr-2026 Rs.286.06
Till 22-Apr - 24-Apr Rs.143.03
برائے کرم مقررہ تاریخ تک بل ادا کر کے بچت کریں۔

Electricity You Have Used آپ کی استعمال شدہ بجلی

	Previous Reading	Current Reading	Units (KWh)	MDI (KW)
Energy	8819	8970	151	

13 Month Usage History 13 مہینے کے استعمال کی تفصیل

Billing & Payment History بلنگ اور ادائیگی کی تفصیل

MM / YY	Billed Amount	Pay-Date	Payment
03/26	2,577.31	16-Mar-26	2577.00
02/26	2,782.51	19-Feb-26	2783.00
01/26	2,780.55	15-Jan-26	2780.00

For billing history, visit KE website. بلنگ کی تفصیل کے لئے KE کی ویب سائٹ دیکھیں۔

Message Board پیغام بورڈ

FCA of XWDISCOs is also being charged to KE consumers as part of Uniform FCA starting from the month of Jun 25

Bill Calculation آپ کے موجودہ بل کا حساب

Reading Date: 03-Apr-26 | No.of Month(s): 1
Bill Charge Mode: NORM | Installation Date: 20-Oct-1986

	Units	Rate	Amount
Fixed Charges	3.00	300.0000	900.00
Variable Upto 100 Units (Protected)	100.00	10.5400	1,054.00
Variable Upto 200 Units (Protected)	51.00	13.0100	663.51
Uniform Quarterly Adjustment	151.00	0.3504	52.91
FCA -Feb-26	88	1.4235	125.27
Additional Surcharge (PHL)	151.00	0.4300	64.93
Electricity Charges			**2,860.62**
Electricity Duty			29.41
Sales Tax u/s 3(1)*			520.21
MUCT (KMC)			20.00
Taxes and Duties			**569.62**
Electricity Charges for current month			**3,430.24**
Opening Balance			**2,576.47**
Payments/Adjustment			-2,577.00
Previous Dues			**-0.53**
Remaining Dues			**-0.53**
Current Bill Amount			3,430.24
Amount Payable within Due Date			**3,430**
Late Payment Surcharge (5%)			143.03
Late Payment Surcharge (10%)			286.06
Amount Payable from 22-Apr - 24-Apr with 5% LPS			**3,574**
Amount Payable after 24-Apr-26 with 10% LPS			**3,716**

* Sales Tax Act, 1990- ** Income Tax Ordinance, 2001

83.39 %

16.61 %

ENERGY CHARGES 2,860.62

TAXES AND DUTIES 569.62

This electricity bill is issued solely for the purpose of recovering charges for electricity consumption and applicable taxes. The inclusion of consumer name and address in this bill does not constitute or imply proof of ownership / title to the property.

بجلی کا یہ بل صرف بجلی کے استعمال اور لاگو ہونے والے ٹیکسز کی وصولی کے مقصد کے لیے جاری کیا گیا ہے۔ اس بل میں صارف کا نام اور پتہ شامل ہونے کا مطلب یا ثبوت ہرگز نہیں ہے کہ صارف متعلقہ جائیداد کا مالک ہے یا اسے اس کی ملکیت حاصل ہے۔

KE

Customer Name [redacted]

Account Number

Contract Number

Invoice Number [redacted]

Due Date 21-Apr-2026

3,430	22-Apr - 24-Apr 3,574	After 24-Apr 3,716

Bank Clearance / Signature

5.7 Correct JSON for the sample bill

This completed example uses numbers for numeric fields and preserves the printed values.

```json
{
  "provider": "KE",
  "tariff": "A1-R",
  "sanctioned_load_kw": 3,
  "bill_month": "2026-04",
  "reading_date": "2026-04-03",
  "issue_date": "2026-04-07",
  "due_date": "2026-04-21",
  "previous_reading": 8819,
  "current_reading": 8970,
  "units_consumed": 151,
  "charges": [
    {"type": "fixed", "amount": 900},
    {"type": "energy", "amount": 1054},
    {"type": "energy", "amount": 663.51},
    {"type": "quarterly_adjustment", "amount": 52.91},
    {"type": "fpa", "amount": 125.27},
    {"type": "surcharge", "amount": 64.93}
  ],
  "total_charges": 2860.62,
  "taxes": [
    {"type": "electricity_duty", "amount": 29.41},
    {"type": "gst", "amount": 520.21},
    {"type": "municipal_tax", "amount": 20}
  ],
  "total_taxes": 569.62,
  "current_bill": 3430.24,
  "arrears": -0.53,
  "payable_within_due_date": 3430,
  "payable_after_due_date": 3716
}
```

5.8 Key points from the example

- The bill has two energy-charge lines, so `charges` includes two `energy` entries.
- The printed previous dues are a credit of PKR 0.53, so `arrears` is `-0.53`.
- The bill shows more than one late-payment amount. The highest, PKR 3,716, is returned.

5.9 How the JSON looks in level1.csv

Your solution writes each bill's JSON into `level1.csv` as a single line. Inside the quoted CSV cell, every quotation mark is doubled. This is the start of the row for `KESC_0008`:

```csv
bill_id,json
KESC_0008,"{""provider"":""KE"",""tariff"":""A1-R"",""sanctioned_load_kw"":3, ... }"
```

The row is shortened here. The real row continues on the same line with every field from section 5.7.

6. Level 2: Answer customer questions

In Level 2, your solution answers customer questions about each bill in **plain English**, using only information on the bill. Your solution must generate one text answer for every row in `level2.csv`. All 10 test bills have the same 12 questions, so that is **120 answers** in total.

6.1 Answer rules

Follow these rules for every answer.

- **Get the numbers right.** Averages, differences, counts, percentages, and estimates are checked against the bill. Using code for calculations is recommended.
- **Ground each answer in the bill.** Do not use outside tariff or rate information. If the bill does not show a needed value, say so instead of guessing.
- **Label estimates.** Say that a value is an estimate and explain how you calculated it.
- **Explain ambiguous interpretations.** If a question has more than one reasonable meaning, state the meaning you used or answer both.
- **Exclude personal identifiers.**
- **Answer in English.** Write every answer in English, even when parts of the bill are in Urdu.

6.2 Sample questions and answers

The test questions are released at **12:30 PM** in `test/csv/level2.csv`. The examples below use training bill KESC_0008. **The test questions will be different**, but the same answer rules apply.

Bill	Question	Answer
KESC_0008	How much of my bill is taxes?	Taxes and duties total PKR 569.62 out of your current bill of PKR 3,430.24, or about 16.6%. The bill shows a split of 83.39% energy charges and 16.61% taxes. The tax lines are sales tax (PKR 520.21), electricity duty (PKR 29.41), and MUCT (KMC) (PKR 20.00).
KESC_0008	How many months in my history went above 200 units?	Two of the 12 previous months exceeded 200 units: July 2025 (239) and August 2025 (203). The current month used 151 units, so the count is two whether or not the current month is included.

7. What to submit

Submit **one ZIP file** with the structure below. **The ZIP must be 15 MB or smaller.** The `output/` and `source/` folders and their listed files are required. The `demo/` folder is optional.

```text
your-submission.zip
├── output/
│   ├── level1.csv          columns: bill_id, json
│   └── level2.csv          columns: bill_id, question_id, question, answer
├── source/
│   ├── README.md           how to run the project, and how AI was used
│   ├── .env.example        the environment variables your code needs,
│   │                       with API keys left as placeholders
│   └── ...                 your project code
└── demo/                   optional, but strongly encouraged
    └── demo.mp4            a working demo of your solution (.mp4, .mov or .webm)
```

7.1 Result files (CSV)

Your solution must generate both completed CSV files from the supplied templates. **Results edited manually are not accepted.**

File	Required contents
`output/level1.csv`	10 rows. Your solution keeps `bill_id` and writes the bill's JSON object, on one line, into `json`.
`output/level2.csv`	120 rows. Your solution keeps `bill_id`, `question_id`, and `question`, and writes its text answer into `answer`.

7.2 CSV format rules

Follow these rules so your CSV files can be read and scored.

- Keep the original column names and row order. Do not add or remove rows.
- Do not change `bill_id`, `question_id`, or `question`.
- Save as UTF-8. Quote cells that contain commas, quotes, or line breaks. Use a CSV library to handle this.
- In `level1.csv`, keep each JSON object on one line. Inside its quoted CSV cell, double every quotation mark.

Section 5.9 shows how a Level 1 row looks in `level1.csv`.

7.3 What makes a submission valid

Include both CSV files and all project code. A submission missing result files or code files will not be considered.

7.4 Source code and README

Include your project code in `source/`, together with:

- **`README.md`** with setup steps, dependencies, and exact commands to run the project on a folder of bill images and generate both CSV files.
- An **AI usage** section in the README listing models, APIs, services, and any AI tools used to write the code. Write "none" for any category you did not use.
- **`.env.example`** listing every environment variable the code reads. Use the actual non-secret configuration values, including the model provider and model name. Replace API keys and other secrets with placeholders.
- **Do not include API keys or your real `.env` file.**

7.5 Runtime

- Use Python 3.11 or later, or Node.js 22 or later.
- State your version in the README, for example "Python 3.12".
- Include requirements.txt with exact versions (package==x.y.z), or package.json with package-lock.json.

7.6 Environment variables

These names make the model configuration easy to find. They are recommended, not mandatory.

Variable	Purpose	Example
`MODEL_PROVIDER`	Model provider	`anthropic`, `google`, or `openai`
`MODEL_NAME`	Model ID used by the provider's API	`claude-haiku-4-5`
`API_KEY`	Provider API key	`your-api-key-here`

Example `.env.example`:

```env
MODEL_PROVIDER=anthropic
MODEL_NAME=claude-haiku-4-5
API_KEY=your-api-key-here
```

7.7 Demo video (optional)

A demo is strongly encouraged and can earn **up to five bonus points**. Put it in the `demo/` folder as `demo.mp4`, `demo.mov` or `demo.webm`. It must be **three minutes or shorter**, and small enough that the whole ZIP stays within **15 MB**.

Record your solution working from start to finish. Audio and captions are not required, but all on-screen content must be readable. Show:

- Starting the solution from its command line or user interface.
- Uploading or selecting a bill, with the bill image visible.
- The Level 1 extracted fields for that bill.

- Level 2 answers to a few questions about the same bill.
- The solution generating `level1.csv` and `level2.csv` for the test bills.

7.7.1 How to record your screen

You do not need special software. Use the recorder built into your system:

System	How to record	Saved as
Windows	Press `Win + Shift + R` to open the Snipping Tool recorder, select the area and click **Start**. Click **Stop** when you are done, then save the video.	`.mp4`
macOS	Press `Cmd + Shift + 5`, choose **Record Entire Screen** or **Record Selected Portion**, then click **Record**. Stop the recording from the menu bar.	`.mov`
Linux	Press `Print Screen`, switch to video mode, select the area and click **Record**. Stop the recording from the top bar.	`.webm`

7.8 Final checklist

You can submit only once. Re-submission is not allowed, so check each item before uploading:

- Both CSV files were generated by your solution, are valid, and have no empty `json` or `answer` cells.
- The ZIP contains your project code, a clear README, and `.env.example`.
- The README includes setup commands, run commands, and the AI usage section.
- No real API keys, other secrets, or `.env` files are included.
- If included, the demo plays correctly and is no longer than three minutes.
- The ZIP is 15 MB or smaller.
- You have opened the ZIP and checked its folders and files.

Upload before **1:30 PM** at https://build.womentechquest.com/submit.

8. Scoring

Your submission is assessed on its results and implementation. The base score is **100 points**, with **up to five additional points** for the optional demo.

Area	Points	What is assessed
Extraction accuracy	30	Correct Level 1 fields across the test bills. Text and dates must match exactly; numbers are accepted within ±1. Missing required fields and invented values reduce the score.
Answer quality	30	Scored by an LLM judge: correct, specific Level 2 answers grounded in the bill, with accurate numbers and clear statements when information is unavailable.
Engineering quality	15	Clean, well-organized code that handles errors gracefully.
AI design	15	Thoughtful, reliable use of AI, with the model used where it adds the most value.
Completeness and reproducibility	10	A working end-to-end solution that someone can easily set up and run.
Base total	**100**	
Demo video bonus	**Up to 5**	An optional short recording that shows the solution working from start to finish.

9. AI models and API keys

This competition requires the use of large language models (LLMs). Set up an API key for one of the models below at the start of the event.

9.1 Allowed models

To keep the competition fair, you may use only the following models. **Using any other model leads to disqualification.** You will be asked to state the model you used when you submit.

Model	Model ID	Rate limits	Price per 1M tokens (input / output)	Access
Claude Haiku 4.5 Anthropic	`claude-haiku-4-5`	1,000 requests/min 2M input + 400K output tokens/min (new accounts may start lower)	$1.00 / $5.00	**Paid** Bring your own credit
Gemini 3.5 Flash-Lite Google	`gemini-3.5-flash-lite`	Free tier: 15 requests/min, 250K tokens/min, 500 requests/day	Free tier: $0 Paid: $0.30 / $2.50	**Free** Google AI Studio

Model	Model ID	Rate limits	Price per 1M tokens (input / output)	Access
Gemini 3.1 Flash-Lite Google	`gemini-3.1-flash-lite`	Free tier: 15 requests/min, 250K tokens/min, 500 requests/day	Free tier: $0 Paid: $0.25 / $1.50	**Free** Google AI Studio
GPT-4.1 mini OpenAI	`gpt-4.1-mini`	500 requests/min, 10,000 requests/day, 200K tokens/min	$0.40 / $1.60	**Paid** Bring your own credit
GPT-5 mini OpenAI	`gpt-5-mini`	500 requests/min, 500K tokens/min	$0.25 / $2.00	**Paid** Bring your own credit

Prices and limits are taken from each provider and may change.

9.2 Credits

Choose a paid provider with your own credit, or use a free Gemini key.

- **Using OpenAI or Anthropic?** Bring your own credit. Roughly $5–$10 should be enough for the competition.
- **Prefer a free option?** Google AI Studio offers a free API key for Gemini. Follow the steps in section 9.3.

9.3 How to get a free API key

No credit card is required. This key works with the Google AI Studio (Gemini) models: `gemini-3.5-flash-lite` or `gemini-3.1-flash-lite`.

1. Go to aistudio.google.com.
2. Sign in with your Google account.
3. Click **Get API key** (keys icon, bottom left).
4. Click **Create API key** and choose **Default Gemini Project**.
5. Copy the key.

Tip: Set up your API key first and make one test call with it, so you do not lose build time on setup.

9.4 Keep your API key private

Do not share your key or put it directly in your code. Store it in a `.env` file, and do not include that file in your submission (see section 7.4).